

Scope Plan — PII Search Engine (IntelX-style MVP)

Context

The customer is delivering a **self-hosted PII search engine** to their own customers. Each customer runs a private instance (we sell + optionally install for an extra fee). The product lets the operator ingest large dumps of leads (phones, emails, usernames, ID cards, plus open-ended PII) and query them in milliseconds, with tag-based organization and AI-assisted natural-language queries.

Goal of this document: a **product spec** the developer can read end-to-end and build against. Technical choices and complete features overview

1. What data is indexed

1.1 The lead (record) model

A **lead** is one row of information about a person/account. Every lead **must contain at least one** of the four **identity fields**:

Identity field	Notes
phone	Stored normalized (digits only + country code) AND original(to discuss). Country-code aware.
email	Stored normalized (lowercased) AND original(to discuss). Domain & local-part queryable separately(no idea if affets the design very heavy).
username	Free string.
id_card	National ID / DNI / passport-style. Free string, no per-country validation in MVP.

If a batch row has none of the four → **rejected at ingest** with a clear error.

1.2 Other fields

- A **fixed canonical set** of common PII fields the engine knows by name and indexes natively: `full_name`, `first_name`, `last_name`, `dob`, `address`, `city`, `country`, `zip`, `ip`, `password`, `password_hash`, `gender`. (Final list: developer to confirm with operator.)
- Open extras**: any additional field provided by the batch is stored and queryable as a generic key/value extra. We want to prioritize flexibility.
- Implication**: To design the storage so canonical fields are columnar/typed (fast) and extras live in a flexible structure (slower but searchable). Mixed model. (This is just a suggestion)

1.3 Tags

- Tags are **first-class objects with metadata**, not just strings.
- Tag fields: `name` (e.g. `Ticketmaster-VM`), `source_url`, `breach_date`, `type` (e.g. `LOGIN`, `VM`, `LEAK`), `notes`.
- A lead can have **N tags**. A tag can be applied to **N leads**. Many-to-many.
- Tags are global to the instance. Operator creates/edits/deletes tags freely from the UI.

1.4 Identity merging — OPEN QUESTION, flagged for the developer

When the same phone/email appears across two batches, two options are on the table:

- (A) Keep batches separate, link on query**. Each row is independent. A query for phone X returns N hits with all their tags. Auditable, simple.
- (B) Hybrid**. Underlying data stays per-batch (auditable), but UI can collapse hits into a “merged profile” view on demand.

The operator wants the developer's input here — there may be a big architectural advantage one way or the other (entity resolution, dedup compression, query cost). the developer must d**Decide this together before building**. Default if undthe developer must decided: **(B) Hybrid** — store per-batch, merge on view.

2. How data is ingested

2.1 Supported input formats

- Structured**: CSV, JSON, JSONL, SQL dumps (`INSERT INTO ...` parsing).
- Semi-structured**: TXT with `:`, `|`, `;` and other delimiters; combo lists (`email:password`, `user:pass:ip`, etc).
- Compressed archives**: `.zip`, `.gz`, `.7z`, `.tar.gz` — engine unpacks, processes every contained file, treats each as a sub-batch (or one merged batch — operator's choice at upload).
- Out of MVP**: PDFs, free-text logs, scraped HTML.

2.2 Two ingestion paths

Both write into the same store; both apply the same identity-field requirement and tag rules.

A) UI-driven bulk ingest

- Operator uploads file/archive.
- Engine **auto-detects** structure: format, delimiter, columns, candidate identity fields.
- Operator sees a **preview**: ~20-50 sample rows in their parsed structured form, plus the proposed column → field mapping.
- Operator can **manually remap columns** (drag/edit field assignments, mark columns to skip, mark a column as “extra/custom field name X”).
- Operator selects **tags to apply** to the entire batch (existing tags or create new ones inline).
- Operator confirms → ingest runs in background; progress shown.
- Validation: rows missing all 4 identity fields are dropped, count reported.

B) API ingest (single-row or small batches)

- HTTP endpoint: `POST /api/index` →JSON body with one or many leads + optional tag IDs/names.
- Same identity-field requirement enforced.
- Auth: per-instance API key (single token in MVP; multi-key later).
- Use case: scripts, integrations (Checkers, VMs).

2.3 Bulk tag update (no new data)

- Operator uploads a list (CSV) containing **at least one identity field per row**. Then we select what TAGs we want to include.
- Matching: **normalized** (phone digits-only, email lowercased, etc.) against existing leads.
- Output: tag is applied to all matched leads + a downloadable **CSV of unmatched rows** so the operator knows what didn't land. (maybe in case they are not added, we could add them? But still, I would add this later on to verify normalize methodologies work perfect)

2.4 Ingest-time data hygiene

- Normalize phone/email at write time, store both raw and normalized (if normalization really worked well there wasn't a need to actually store original. No idea about to how extra size original storage would causing.).
- Skip duplicate identical rows within the same batch
- Cross-batch dedup behavior depends on the **identity-merging decision** in §1.4.

3. How data is accessed (queries)

3.1 Structured queries

Operator can query by any field with these match modes:

Match mode	Example	Notes
Exact	<code>email:john@outlook.com</code>	Default.
Prefix wildcard	<code>username:john*</code>	
Suffix wildcard	<code>phone:*7434</code>	The example the operator called out.
Substring	<code>email:*outlook*</code>	Heavier; needs n-gram index.
Component	<code>email.domain:outlook.com</code> , <code>phone.country:+34</code>	Engine-aware splits. Faster than wildcards.
Tag filter	<code>tag:Ticketmaster-VM</code> , <code>tag.type:LOGIN</code> , <code>tag.breach_date:2024</code>	Combinable with anything above.
Boolean compose	<code>email:*@outlook.com AND tag:Ticketmaster-VM</code>	AND/OR/NOT.

- Regex is **out of MVP** (flagged for later; useful but slow).
- Results are paginated; default page size 50; max page size 1000.
- Every result row shows: all canonical fields populated, all extras, all tags, source batch, ingest date.

3.2 AI natural-language query

- Single text box: “I want all emails @outlook.com tagged Ticketmaster-VM registered in 2024.”
- AI **translates NL** → **structured filter** (the same DSL as §3.1). No semantic/embedding search in MVP.
- The translated filter is **shown to the operator before executing** so they can tweak/learn the syntax.
- LLM choice is the developer's (local model preferred for self-hosted privacy; API model acceptable if operator opts in).

3.3 Export

- Any query result set can be exported to **CSV** or **JSONL**.
- Background job for large exports; download link when ready.

3.4 Out of MVP (planned future)

- Audit log (who queried what, when).
- Per-record deletion / takedown endpoint.
- Multi-user roles.
- Regex queries.
- Semantic search on free-text fields.

4. APIFICATION of outputs

4.1 As well we have input indexing endpoints. It suit perfect to have output endpoints. The entire API structure should be protected by a single API-Key (no need for multi api-key layer)

5. How data is stored & operated

This section is **product-level constraints**, not architecture decisions — those belong to the developer. But these constraints must hold:

- Self-hosted, single-tenant per instance**. No SaaS, no cross-customer data. All infra open-source.
- Scale planning**: MVP runs comfortably on one beefy server (~10–100 GB to ~1 TB). Architecture must shard cleanly so the same product can be deployed to a customer with **100+ TB** without a rewrite. Developer chooses the engine (likely candidates: ClickHouse, OpenSearch/Elastic, Meilisearch, Tantivy, or a hybrid) — but the choice must demonstrably handle the scale path.
- Query speed target**: sub-second for typical filtered queries on the MVP scale; sub-5-second on multi-TB shards. Wildcard `*substring*` queries are allowed to be slower.
- Data Compression**: prefer engines/formats with native columnar compression (ClickHouse-style, Parquet-backed, ...) — operator explicitly wants disk space minimized (true --> 1, false --> 0, things like that)
- Deployment**: single `docker compose` up (or equivalent) for the operator to install on their own server. Documented hardware sizing per scale tier. Still up to the developer to set it its own way.
- No external dependencies the customer can't self-host**. Anthropic/OpenAI for the AI query is acceptable tho.

6. Open questions for the developer (to resolve with operator before build)

- Identity merging model** (§1.4) — A vs hybrid B. Architectural impact.
- Final canonical field list (§1.2) — confirm the ~12 fields above cover 90% of real batches.
- Storage engine choice + sharding plan — must justify the path from 100 GB → 100 TB.
- Hardware sizing reference for the self-hosted install doc.

7. Verification

End-to-end flows the operator should be able to perform in a fresh install:

I trust your criteria + I'm the one who will be pushing for tests and reporting possible whenever I find those.

8. Explicitly out of scope for MVP

- Multi-user accounts, roles, audit log
- Per-record deletion endpoint (GDPR-style takedown)
- PDF / unstructured text extraction
- Regex queries
- Semantic search on free-text fields
- Cross-instance federation / sharing between customers
- Mobile UI (responsive web is enough)